

EXPERIMENT 2

C Programming Environment

Circle Area Program and Compilation Stages using -save-temps

LAB REPORT

STUDENT NAME: Rudra Pratap Singh

SAP ID:590032811

COURSE: B.Tech CSE(2026-30)

Report focus: Retain and inspect the preprocessed (.i), assembly (.s), and object (.o) files produced during compilation; then execute ./a.out in Linux and verify the computed area.

Aim

To write a C program that calculates the area of a circle, demonstrate its compilation stages using `cc/gcc` with the `-save-temps` option, inspect the intermediate files in a Linux editor, and execute the generated `a.out` file using `./a.out`.

Learning outcomes

1. Identify the input and output associated with preprocessing, compilation, assembly, linking, and execution.
2. Use `-save-temps` to retain the intermediate `.i`, `.s`, and `.o` files that GCC normally deletes.
3. Relate selected lines in the intermediate files to statements in the original C program.
4. Validate program output by comparing it with a manual area calculation.

Requirements

Item	Example	Purpose
Compiler	<code>cc</code> or <code>gcc</code> (GCC)	Compiles C and retains intermediate files
Editor / terminal	<code>vi</code> / <code>gedit</code> and Linux shell	Opens intermediate files and runs commands
Source file	<code>circle_area.c</code>	Contains the circle-area program

Theory

Area calculation

For a circle of radius r , the area A is calculated as $A = \pi \times r^2$. The program stores the radius as a double-precision value, multiplies it by itself, and then multiplies the result by a constant approximation of π .

Example check: For $r = 7.00$, $A = 3.141592653589793 \times 7.00 \times 7.00 = 153.938\dots$; displayed to two decimal places, the result is 153.94 square units.

Compilation pipeline

`circle_area.c -> circle_area.i -> circle_area.s -> circle_area.o -> a.out`

Stage	Driver option	Input	Retained output	What happens
Preprocessing	-E	.c	.i	Expands headers and macros; handles # directives
Compilation	-S	.i	.s	Translates C into target assembly language
Assembly	-c	.s	.o	Encodes assembly as relocatable machine code
Linking	(driver)	.o + libraries	a.out	Resolves external symbols and creates a runnable Linux program
Execution	N/A	a.out	Program output	./a.out reads the radius and prints the computed area

Source program

The program uses double for the input and result, validates that the input is numeric and non-negative, and prints the area to two decimal places.

Listing 1. Complete source file: circle_area.c

```
01 #include <stdio.h>
02
03 #define PI 3.141592653589793
04
05 int main(void)
06 {
07     double radius;
08     double area;
09
10     printf("Enter radius of circle: ");
11     if (scanf("%lf", &radius) != 1 || radius < 0.0) {
12         printf("Invalid radius.\n");
13         return 1;
14     }
15
16     area = PI * radius * radius;
17     printf("Area of circle = %.2f square units\n", area);
18
19     return 0;
20 }
```

Compilation procedure

5. Save Listing 1 as circle_area.c in a new working directory.
6. Open a terminal in that directory and compile with warnings enabled while retaining temporary files.
7. List the directory contents and confirm that the .i, .s, and .o files were created.
8. Open circle_area.i, circle_area.s, and circle_area.o with vi or gedit and compare their readability.
9. Run ./a.out, enter a radius, and record the displayed area.

Command 1. Linux cc/gcc command that performs all stages and saves intermediates

```
cc -Wall -Wextra -std=c11 -save-temps circle_area.c
```

On a Linux system where cc invokes GCC, this command completes every compilation stage, keeps circle_area.i, circle_area.s, and circle_area.o, and creates the default executable a.out. The command gcc may be used in place of cc. Omitting -o is intentional because the required executable name is a.out.

Expected retained files

File	Type	Interpretation
circle_area.c	C source	Human-readable program written by the programmer
circle_area.i	Preprocessed C	Expanded translation unit after preprocessing
circle_area.s	Assembly	Target-specific assembly instructions and data
circle_area.o	Object	Relocatable machine code plus symbols and relocation records
a.out	Linux executable	Linked program executed from the current directory with ./a.out

Verifying readability in Linux editors

The retained files can be opened directly with vi or gedit. The .i and .s files are text and can be read by a person, although the .i file is long and the .s file requires assembly-language knowledge. The .o file is binary ELF object code, so opening it in a text editor displays control characters or unreadable symbols.

Command 2. Opening all three intermediate files

```
$ vi circle_area.i
$ gedit circle_area.s &
$ vi circle_area.o
```

File	What appears in vi / gedit	Human-readable?
circle_area.i	Expanded C source, header declarations, and substituted macros	Yes - plain text
circle_area.s	Assembly labels, directives, mnemonics, registers, and constants	Yes - assembly text
circle_area.o	ELF binary bytes and control characters	No - binary file

Snippet 1. Linux file-type check confirms text versus binary

```
$ file circle_area.i circle_area.s circle_area.o
circle_area.i: C source, ASCII text
circle_area.s: assembler source, ASCII text
circle_area.o: ELF 64-bit LSB relocatable, x86-64, not stripped
```

Snippet 2. Beginning of circle_area.o when opened in vi

```
"circle_area.o" [noeol] ... bytes
^?ELF^B^A^A^@^@^@ ...
```

Readability conclusion: circle_area.i and circle_area.s are human-readable text files. circle_area.o is machine-oriented binary data and is not meaningfully readable in vi or gedit; utilities such as nm, objdump, readelf, or xxd should be used to inspect it.

Stage-by-stage evidence

Reading note: The following excerpts are deliberately trimmed to the lines that demonstrate each stage. Header expansion makes the full .i file much longer, and exact assembly mnemonics depend on the GCC version, target architecture, operating system, and optimization level.

Stage 1 - Preprocessing (.c to .i)

The preprocessor handles directives beginning with #. It inserts the declarations needed from stdio.h, removes comments, and replaces PI with its numeric definition. The resulting .i file is still C text but is a complete translation unit for the compiler proper.

Equivalent standalone command

```
cc -E circle_area.c -o circle_area.i
```

Excerpt 1. Relevant portion of circle_area.i (trimmed)

```
/* Many declarations expanded from <stdio.h> appear above. */

int main(void)
{
    double radius;
    double area;
    printf("Enter radius of circle: ");
    if (scanf("%lf", &radius) != 1 || radius < 0.0) {
        printf("Invalid radius.\n");
        return 1;
    }
    area = 3.141592653589793 * radius * radius;
    printf("Area of circle = %.2f square units\n", area);
    return 0;
}
```

Evidence: The token PI no longer appears in the calculation; it has been replaced by 3.141592653589793. This is direct evidence that macro expansion occurred.

Stage 2 - Compilation (.i to .s)

The compiler parses and type-checks the preprocessed C, performs target-dependent transformations, and emits assembly language. Because assembly is architecture-specific, the excerpt below is representative of unoptimized GCC output for an x86-64 target rather than a byte-for-byte requirement.

Equivalent standalone command

```
cc -S circle_area.i -o circle_area.s
```

Excerpt 2. Representative arithmetic in circle_area.s

```
.LC_PI:
    .double 3.141592653589793

main:
    pushq   %rbp
    movq    %rsp, %rbp
    subq    $32, %rsp
    # ... calls to printf and scanf omitted ...
    movsd   -8(%rbp), %xmm0
    mulsd   %xmm0, %xmm0
    movsd   .LC_PI(%rip), %xmm1
    mulsd   %xmm1, %xmm0
    movsd   %xmm0, -16(%rbp)
    # ... final printf call and return omitted ...
```

Evidence: The mulsd instructions perform scalar double-precision multiplication: radius x radius, followed by multiplication by the stored pi constant. Register names and instruction order may differ in an actual build.

Stage 3 - Assembly (.s to .o)

The assembler converts the textual instructions into binary machine-code sections. It also builds a symbol table and relocation entries. The .o file is not yet a complete program because references such as printf and scanf still need to be resolved during linking.

Equivalent standalone command

```
cc -c circle_area.s -o circle_area.o
```

Excerpt 3. Representative symbol-table view of circle_area.o

```
$ nm circle_area.o
0000000000000000 T main
                 U printf
                 U scanf
```

Interpretation: T marks main as code defined in this object file. U marks library functions that are currently undefined here and must be supplied by the link step. On some GNU/Linux systems the scanf symbol may be shown as __isoc99_scanf.

Stage 4 - Linking (.o to executable)

The cc/GCC driver invokes the linker, combines circle_area.o with startup code and the required C runtime libraries, resolves external symbols, and writes the default Linux executable a.out. The original -save-temps command performs this step automatically after creating the object file.

Equivalent standalone link command

```
cc circle_area.o
```

Checkpoint: A successful link produces a.out and returns to the shell without an error message. A missing a.out file indicates that an earlier stage or the link itself failed.

Execution

Linux does not normally search the current directory for commands, so the relative path prefix ./ is used to run the generated a.out file.

Command 3. Executing a.out from the Linux command line

```
$ ls -l a.out
$ ./a.out
```

Output 1. Sample execution with radius 7

```
$ ./a.out
Enter radius of circle: 7
Area of circle = 153.94 square units
```


Validation

The displayed results are rounded to two digits after the decimal point by the %.2f conversion. The following cases provide a compact functional check of the formula and the input-validation branch.

Test	Input	Expected output	Status
Zero radius	0	0.00 square units	Pass if matched
Fractional radius	2.5	19.63 square units	Pass if matched
Integer radius	7	153.94 square units	Pass if matched
Negative radius	-3	Invalid radius.	Pass if rejected
Non-numeric input	abc	Invalid radius.	Pass if rejected

Observations

10. The .i file is substantially larger than the original source because the contents and declarations reached through stdio.h are expanded into the translation unit.
11. The #define directive is absent from the relevant preprocessed expression because the macro name PI has already been substituted.
12. The .s file is readable text but target-specific; optimization options can change its structure without changing program behaviour.
13. The .o file contains machine code but still records unresolved references to library routines. Those references are resolved by the linker.
14. -save-temps does not stop compilation at an intermediate stage. It completes the full build and keeps the normally temporary files for inspection.

Precautions and common errors

15. Use %lf with scanf when reading a double value.
16. Keep the output basename distinct when comparing multiple builds, otherwise saved intermediate files can be overwritten.
17. Treat assembly excerpts as target-dependent evidence; do not expect identical registers or labels on every computer.
18. Check the compiler diagnostics before running the executable. Warnings often reveal type or format-string mistakes.
19. Reject a negative radius because it is not physically meaningful for this calculation.

Result

Completed: The C program was compiled with cc/GCC using -save-temps, exposing the preprocessing (.i), compilation (.s), assembly (.o), and linking stages. The .i and .s files were readable as text, the .o file was confirmed as binary, and ./a.out accepted a radius of 7 and produced 153.94 square units.

Conclusion

This experiment connects a short C source program to the concrete files created by the compiler toolchain. Inspecting the retained outputs shows that compilation is a sequence of transformations rather than a single opaque action: directives and macros are resolved first, C is translated to assembly, assembly becomes relocatable object code, and the linker produces the executable that is finally loaded and run.